

A decorative graphic on the left side of the slide, consisting of a network of white lines and small circles on a dark blue background, resembling a circuit board or a neural network.

DYNAMIC PROGRAMMING INTRODUCTION

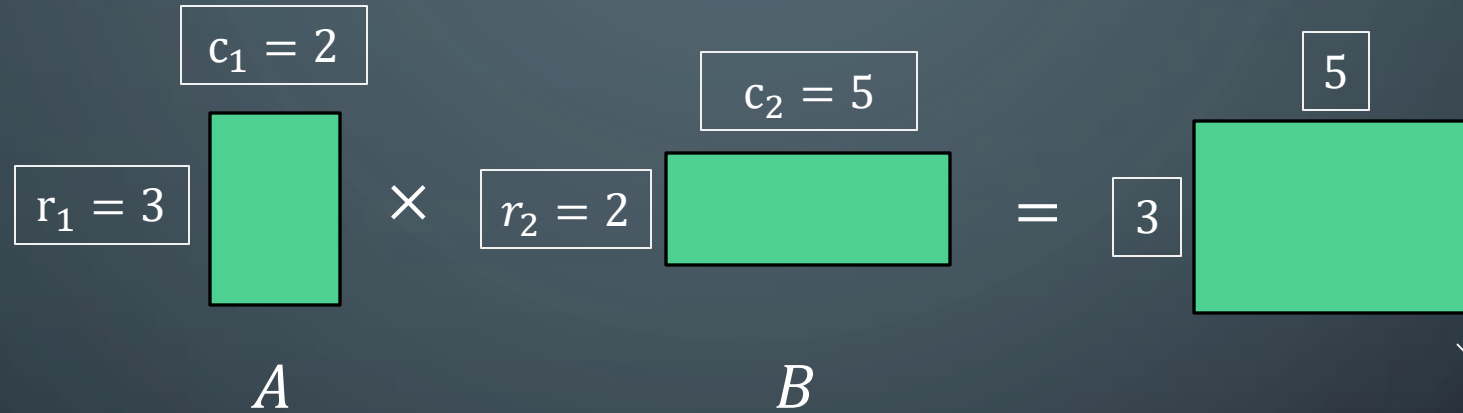
DATA STRUCTURES AND ALGORITHMS 2
MARK FLORYAN

DYNAMIC PROGRAMMING AND GREEDY APPROACH

- TOPICS:
 - Intro to Dynamic Programming
 - Memoization
 - Example DP Problem:
 - Log Cutting

MOTIVATING EXAMPLE

How many scalar multiplications are required to multiply matrices A and B in this example?



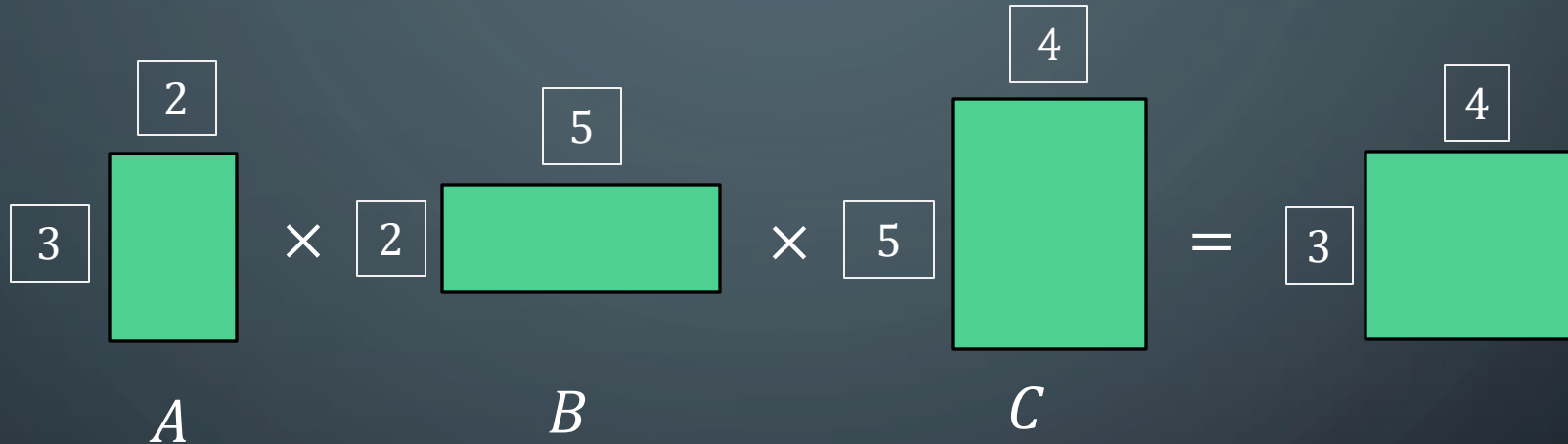
Total is:
 $r_1 c_2 c_1 = (3)(5)(2) = 30$

c_1 multiplications
per element in result

$r_1 c_2$ elements in
result matrix to
compute

TRICKIER EXAMPLE

How many scalar multiplications are required to multiply matrices A , B and C in this example?



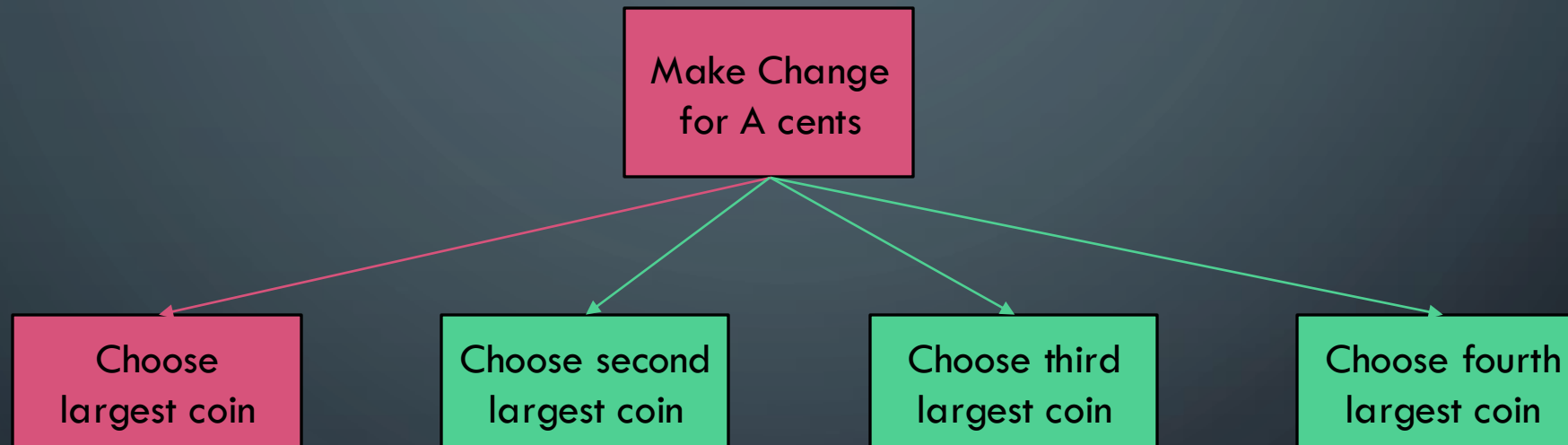
Same formula:

$$(r_1 c_1 c_2) + (r_1 c_2 c_3) = [(3)(2)(5)] + [(3)(5)(4)] = 90$$

Can actually be done in just 64!
Think about why this might be?

DYNAMIC PROGRAMMING AND GREEDY APPROACH

With **Greedy Algorithms**, problem had **optimal substructure** and there was a known choice to make at each step of the problem that guaranteed an optimal solution

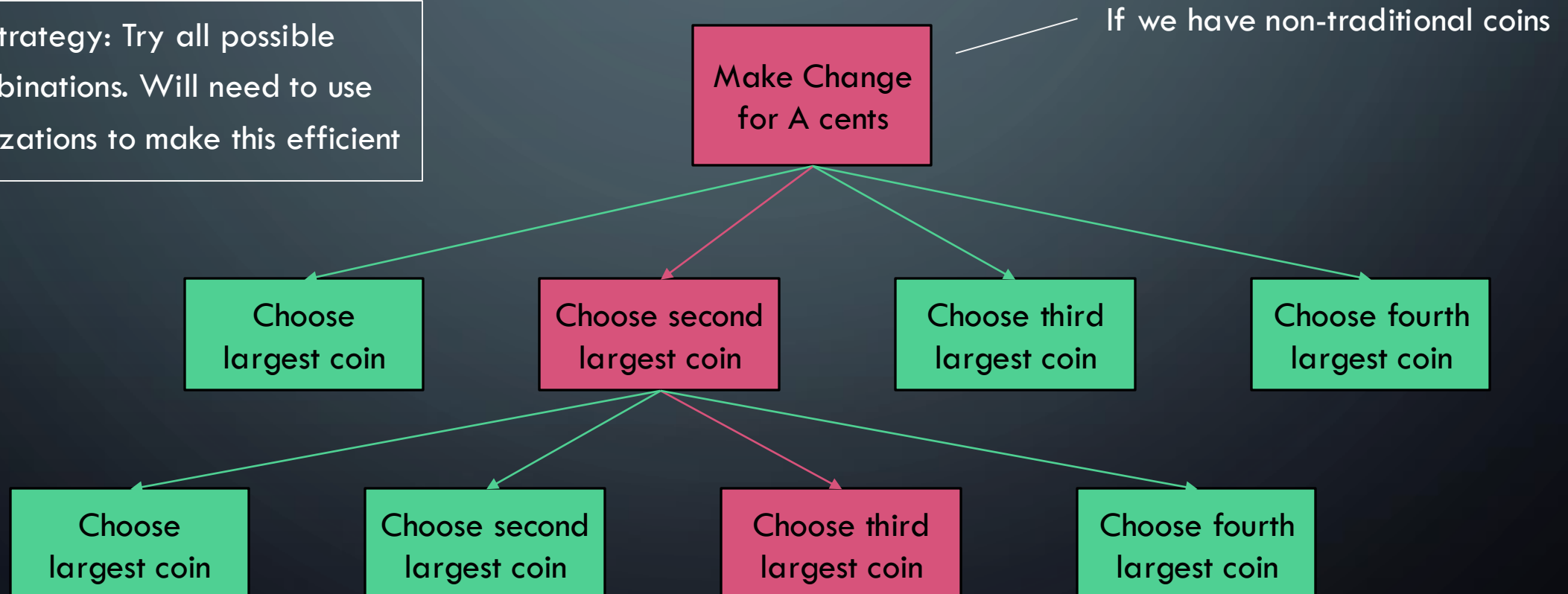


This was ALWAYS the right choice,
never a need to look back

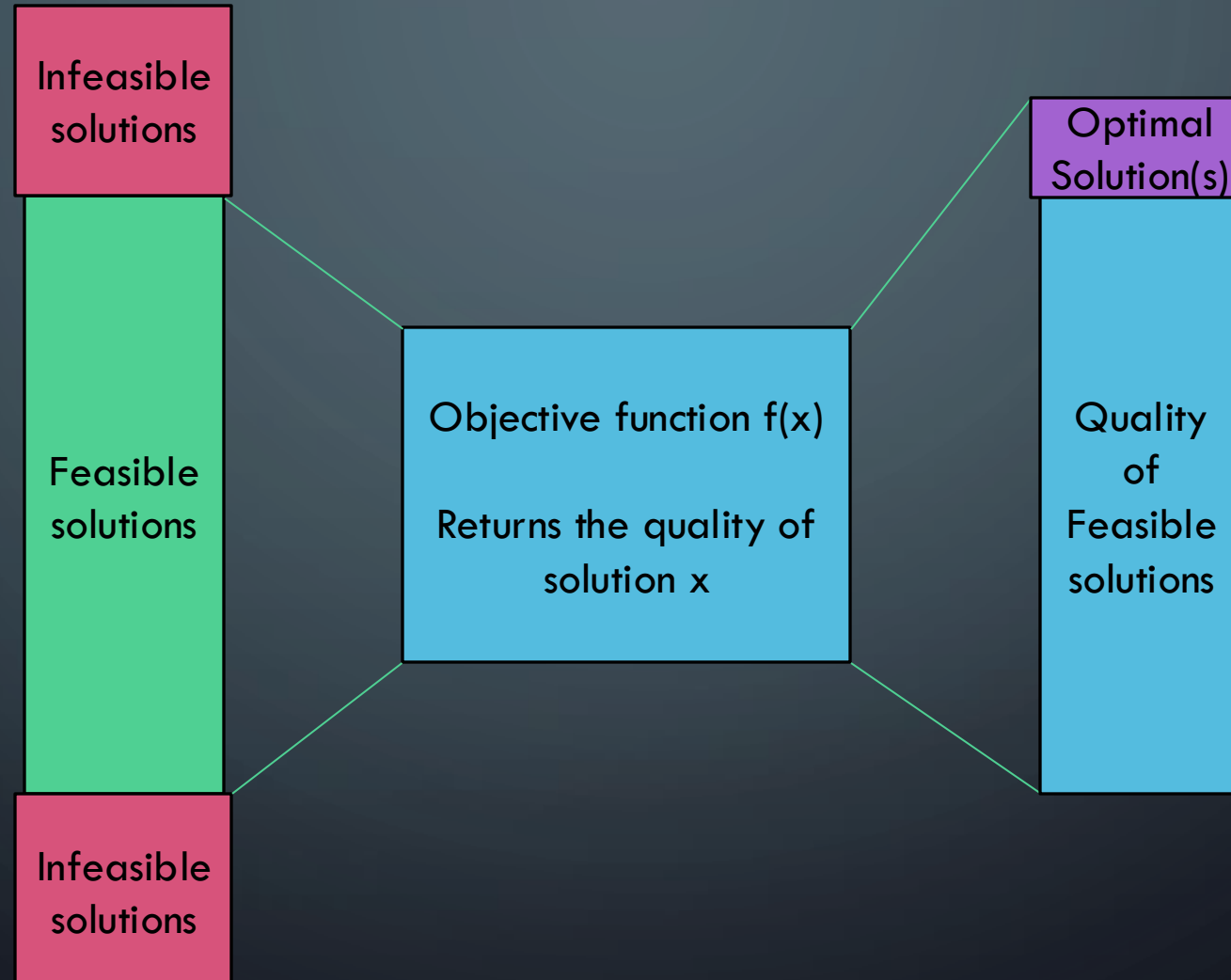
DYNAMIC PROGRAMMING AND GREEDY APPROACH

With **Dynamic Programming**, problem has **optimal substructure** but we don't know which choice at each step will lead to optimal solution

Strategy: Try all possible combinations. Will need to use optimizations to make this efficient



REMINDER: OPTIMIZATION PROBLEMS



A decorative graphic on the left side of the slide, consisting of a network of thin, light blue lines and small circles, resembling a circuit board or a neural network diagram. The lines are vertical and horizontal, with some diagonal connections, and the circles are placed at various points along these lines.

MEMOIZATION

REMEMBER FIBONACCI NUMBERS?

Formula: $f(n) = f(n - 1) + f(n - 2)$

```
long fib(int n) {  
    assert(n >= 0);  
    if ( n == 0 ) return 0;  
    if ( n == 1 ) return 1;  
    return fib(n-1) + fib(n-2);  
}
```

This is a BAD implementation! Repeats function calls repeatedly and runtime is obscenely exponential.

REMEMBER FIBONACCI NUMBERS?

Formula: $f(n) = f(n - 1) + f(n - 2)$

```
long fib(int n) {  
    assert(n >= 0);  
    if ( n == 0 ) return 0;  
    if ( n == 1 ) return 1;  
    return fib(n-1) + fib(n-2);  
}
```

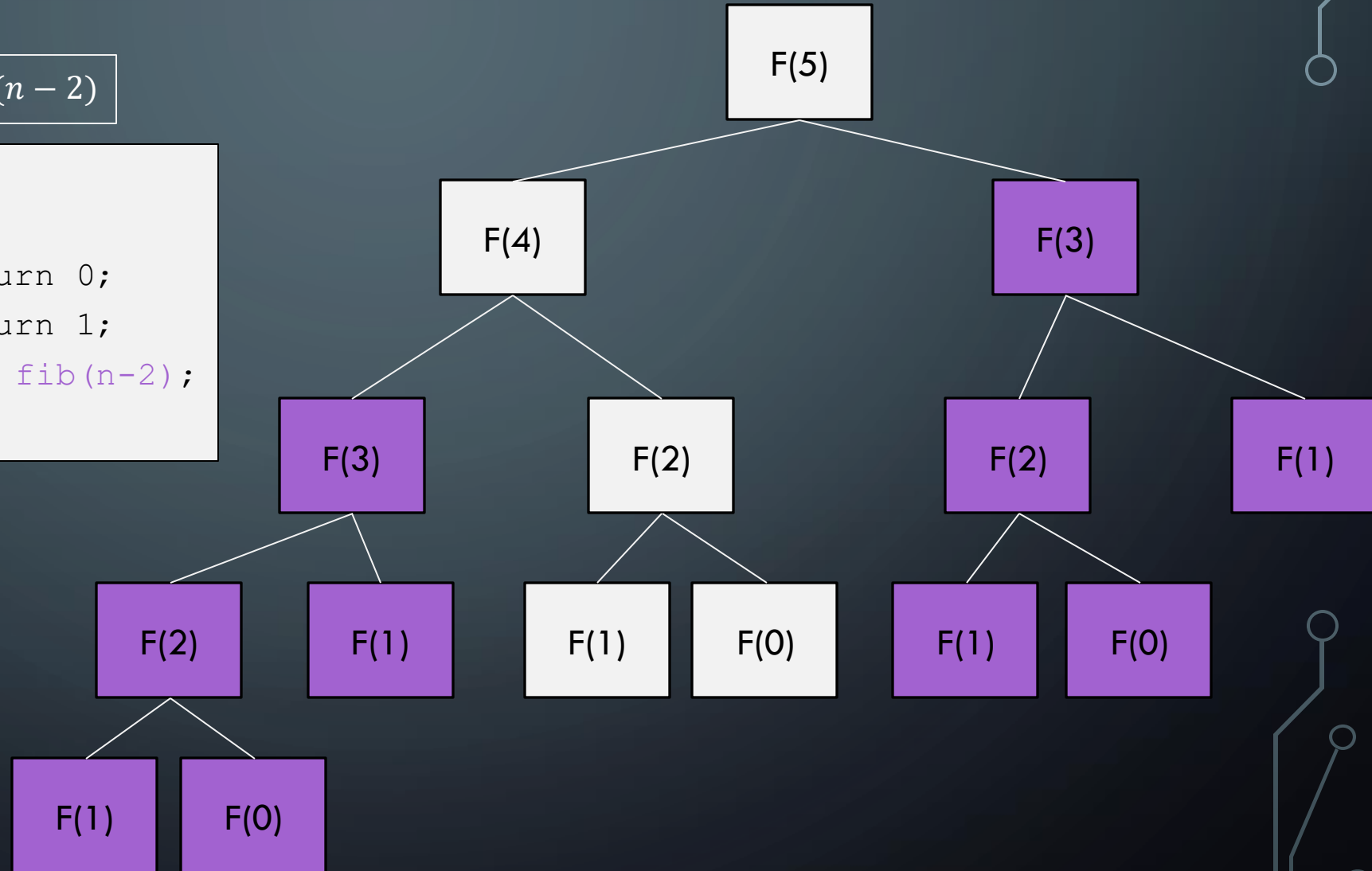
*Entire sub-trees
are repeated!!*

```
graph TD; F5[F(5)] --> F4[F(4)]; F5 --> F3_1[F(3)]; F4 --> F3_2[F(3)]; F4 --> F2_1[F(2)]; F3_1 --> F2_2[F(2)]; F3_1 --> F1_1[F(1)]; F3_2 --> F2_3[F(2)]; F3_2 --> F1_2[F(1)]; F2_1 --> F1_3[F(1)]; F2_1 --> F0_1[F(0)]; F2_2 --> F1_4[F(1)]; F2_2 --> F0_2[F(0)]; F2_3 --> F1_5[F(1)]; F2_3 --> F0_3[F(0)]; F1_1 --> F0_4[F(0)]; F1_2 --> F0_5[F(0)]; F1_3 --> F0_6[F(0)]; F1_4 --> F0_7[F(0)]; F1_5 --> F0_8[F(0)]; F1_6[F(1)] --> F0_9[F(0)]; F1_7[F(1)] --> F0_10[F(0)]; F1_8[F(1)] --> F0_11[F(0)]; F1_9[F(1)] --> F0_12[F(0)]; F1_10[F(1)] --> F0_13[F(0)]; F1_11[F(1)] --> F0_14[F(0)]; F1_12[F(1)] --> F0_15[F(0)]; F1_13[F(1)] --> F0_16[F(0)]; F1_14[F(1)] --> F0_17[F(0)]; F1_15[F(1)] --> F0_18[F(0)]; F1_16[F(1)] --> F0_19[F(0)]; F1_17[F(1)] --> F0_20[F(0)]; F1_18[F(1)] --> F0_21[F(0)]; F1_19[F(1)] --> F0_22[F(0)]; F1_20[F(1)] --> F0_23[F(0)]; F1_21[F(1)] --> F0_24[F(0)]; F1_22[F(1)] --> F0_25[F(0)]; F1_23[F(1)] --> F0_26[F(0)]; F1_24[F(1)] --> F0_27[F(0)]; F1_25[F(1)] --> F0_28[F(0)]; F1_26[F(1)] --> F0_29[F(0)]; F1_27[F(1)] --> F0_30[F(0)]; F1_28[F(1)] --> F0_31[F(0)]; F1_29[F(1)] --> F0_32[F(0)]; F1_30[F(1)] --> F0_33[F(0)]; F1_31[F(1)] --> F0_34[F(0)]; F1_32[F(1)] --> F0_35[F(0)]; F1_33[F(1)] --> F0_36[F(0)]; F1_34[F(1)] --> F0_37[F(0)]; F1_35[F(1)] --> F0_38[F(0)]; F1_36[F(1)] --> F0_39[F(0)]; F1_37[F(1)] --> F0_40[F(0)]; F1_38[F(1)] --> F0_41[F(0)]; F1_39[F(1)] --> F0_42[F(0)]; F1_40[F(1)] --> F0_43[F(0)]; F1_41[F(1)] --> F0_44[F(0)]; F1_42[F(1)] --> F0_45[F(0)]; F1_43[F(1)] --> F0_46[F(0)]; F1_44[F(1)] --> F0_47[F(0)]; F1_45[F(1)] --> F0_48[F(0)]; F1_46[F(1)] --> F0_49[F(0)]; F1_47[F(1)] --> F0_50[F(0)]; F1_48[F(1)] --> F0_51[F(0)]; F1_49[F(1)] --> F0_52[F(0)]; F1_50[F(1)] --> F0_53[F(0)]; F1_51[F(1)] --> F0_54[F(0)]; F1_52[F(1)] --> F0_55[F(0)]; F1_53[F(1)] --> F0_56[F(0)]; F1_54[F(1)] --> F0_57[F(0)]; F1_55[F(1)] --> F0_58[F(0)]; F1_56[F(1)] --> F0_59[F(0)]; F1_57[F(1)] --> F0_60[F(0)]; F1_58[F(1)] --> F0_61[F(0)]; F1_59[F(1)] --> F0_62[F(0)]; F1_60[F(1)] --> F0_63[F(0)]; F1_61[F(1)] --> F0_64[F(0)]; F1_62[F(1)] --> F0_65[F(0)]; F1_63[F(1)] --> F0_66[F(0)]; F1_64[F(1)] --> F0_67[F(0)]; F1_65[F(1)] --> F0_68[F(0)]; F1_66[F(1)] --> F0_69[F(0)]; F1_67[F(1)] --> F0_70[F(0)]; F1_68[F(1)] --> F0_71[F(0)]; F1_69[F(1)] --> F0_72[F(0)]; F1_70[F(1)] --> F0_73[F(0)]; F1_71[F(1)] --> F0_74[F(0)]; F1_72[F(1)] --> F0_75[F(0)]; F1_73[F(1)] --> F0_76[F(0)]; F1_74[F(1)] --> F0_77[F(0)]; F1_75[F(1)] --> F0_78[F(0)]; F1_76[F(1)] --> F0_79[F(0)]; F1_77[F(1)] --> F0_80[F(0)]; F1_78[F(1)] --> F0_81[F(0)]; F1_79[F(1)] --> F0_82[F(0)]; F1_80[F(1)] --> F0_83[F(0)]; F1_81[F(1)] --> F0_84[F(0)]; F1_82[F(1)] --> F0_85[F(0)]; F1_83[F(1)] --> F0_86[F(0)]; F1_84[F(1)] --> F0_87[F(0)]; F1_85[F(1)] --> F0_88[F(0)]; F1_86[F(1)] --> F0_89[F(0)]; F1_87[F(1)] --> F0_90[F(0)]; F1_88[F(1)] --> F0_91[F(0)]; F1_89[F(1)] --> F0_92[F(0)]; F1_90[F(1)] --> F0_93[F(0)]; F1_91[F(1)] --> F0_94[F(0)]; F1_92[F(1)] --> F0_95[F(0)]; F1_93[F(1)] --> F0_96[F(0)]; F1_94[F(1)] --> F0_97[F(0)]; F1_95[F(1)] --> F0_98[F(0)]; F1_96[F(1)] --> F0_99[F(0)]; F1_97[F(1)] --> F0_100[F(0)]; F1_98[F(1)] --> F0_101[F(0)]; F1_99[F(1)] --> F0_102[F(0)]; F1_100[F(1)] --> F0_103[F(0)]; F1_101[F(1)] --> F0_104[F(0)]; F1_102[F(1)] --> F0_105[F(0)]; F1_103[F(1)] --> F0_106[F(0)]; F1_104[F(1)] --> F0_107[F(0)]; F1_105[F(1)] --> F0_108[F(0)]; F1_106[F(1)] --> F0_109[F(0)]; F1_107[F(1)] --> F0_110[F(0)]; F1_108[F(1)] --> F0_111[F(0)]; F1_109[F(1)] --> F0_112[F(0)]; F1_110[F(1)] --> F0_113[F(0)]; F1_111[F(1)] --> F0_114[F(0)]; F1_112[F(1)] --> F0_115[F(0)]; F1_113[F(1)] --> F0_116[F(0)]; F1_114[F(1)] --> F0_117[F(0)]; F1_115[F(1)] --> F0_118[F(0)]; F1_116[F(1)] --> F0_119[F(0)]; F1_117[F(1)] --> F0_120[F(0)]; F1_118[F(1)] --> F0_121[F(0)]; F1_119[F(1)] --> F0_122[F(0)]; F1_120[F(1)] --> F0_123[F(0)]; F1_121[F(1)] --> F0_124[F(0)]; F1_122[F(1)] --> F0_125[F(0)]; F1_123[F(1)] --> F0_126[F(0)]; F1_124[F(1)] --> F0_127[F(0)]; F1_125[F(1)] --> F0_128[F(0)]; F1_126[F(1)] --> F0_129[F(0)]; F1_127[F(1)] --> F0_130[F(0)]; F1_128[F(1)] --> F0_131[F(0)]; F1_129[F(1)] --> F0_132[F(0)]; F1_130[F(1)] --> F0_133[F(0)]; F1_131[F(1)] --> F0_134[F(0)]; F1_132[F(1)] --> F0_135[F(0)]; F1_133[F(1)] --> F0_136[F(0)]; F1_134[F(1)] --> F0_137[F(0)]; F1_135[F(1)] --> F0_138[F(0)]; F1_136[F(1)] --> F0_139[F(0)]; F1_137[F(1)] --> F0_140[F(0)]; F1_138[F(1)] --> F0_141[F(0)]; F1_139[F(1)] --> F0_142[F(0)]; F1_140[F(1)] --> F0_143[F(0)]; F1_141[F(1)] --> F0_144[F(0)]; F1_142[F(1)] --> F0_145[F(0)]; F1_143[F(1)] --> F0_146[F(0)]; F1_144[F(1)] --> F0_147[F(0)]; F1_145[F(1)] --> F0_148[F(0)]; F1_146[F(1)] --> F0_149[F(0)]; F1_147[F(1)] --> F0_150[F(0)]; F1_148[F(1)] --> F0_151[F(0)]; F1_149[F(1)] --> F0_152[F(0)]; F1_150[F(1)] --> F0_153[F(0)]; F1_151[F(1)] --> F0_154[F(0)]; F1_152[F(1)] --> F0_155[F(0)]; F1_153[F(1)] --> F0_156[F(0)]; F1_154[F(1)] --> F0_157[F(0)]; F1_155[F(1)] --> F0_158[F(0)]; F1_156[F(1)] --> F0_159[F(0)]; F1_157[F(1)] --> F0_160[F(0)]; F1_158[F(1)] --> F0_161[F(0)]; F1_159[F(1)] --> F0_162[F(0)]; F1_160[F(1)] --> F0_163[F(0)]; F1_161[F(1)] --> F0_164[F(0)]; F1_162[F(1)] --> F0_165[F(0)]; F1_163[F(1)] --> F0_166[F(0)]; F1_164[F(1)] --> F0_167[F(0)]; F1_
```

Formula: $f(n) = f(n - 1) + f(n - 2)$

```
long fib(int n) {
    assert(n >= 0);
    if ( n == 0 ) return 0;
    if ( n == 1 ) return 1;
    return fib(n-1) + fib(n-2);
}
```

*Entire sub-trees
are repeated!!*



REMEMBER FIBONACCI NUMBERS?

Formula: $f(n) = f(n - 1) + f(n - 2)$

```
long fib(int n) {  
    assert(n >= 0);  
    if ( n == 0 ) return 0;  
    if ( n == 1 ) return 1;  
    return fib(n-1) + fib(n-2);  
}
```

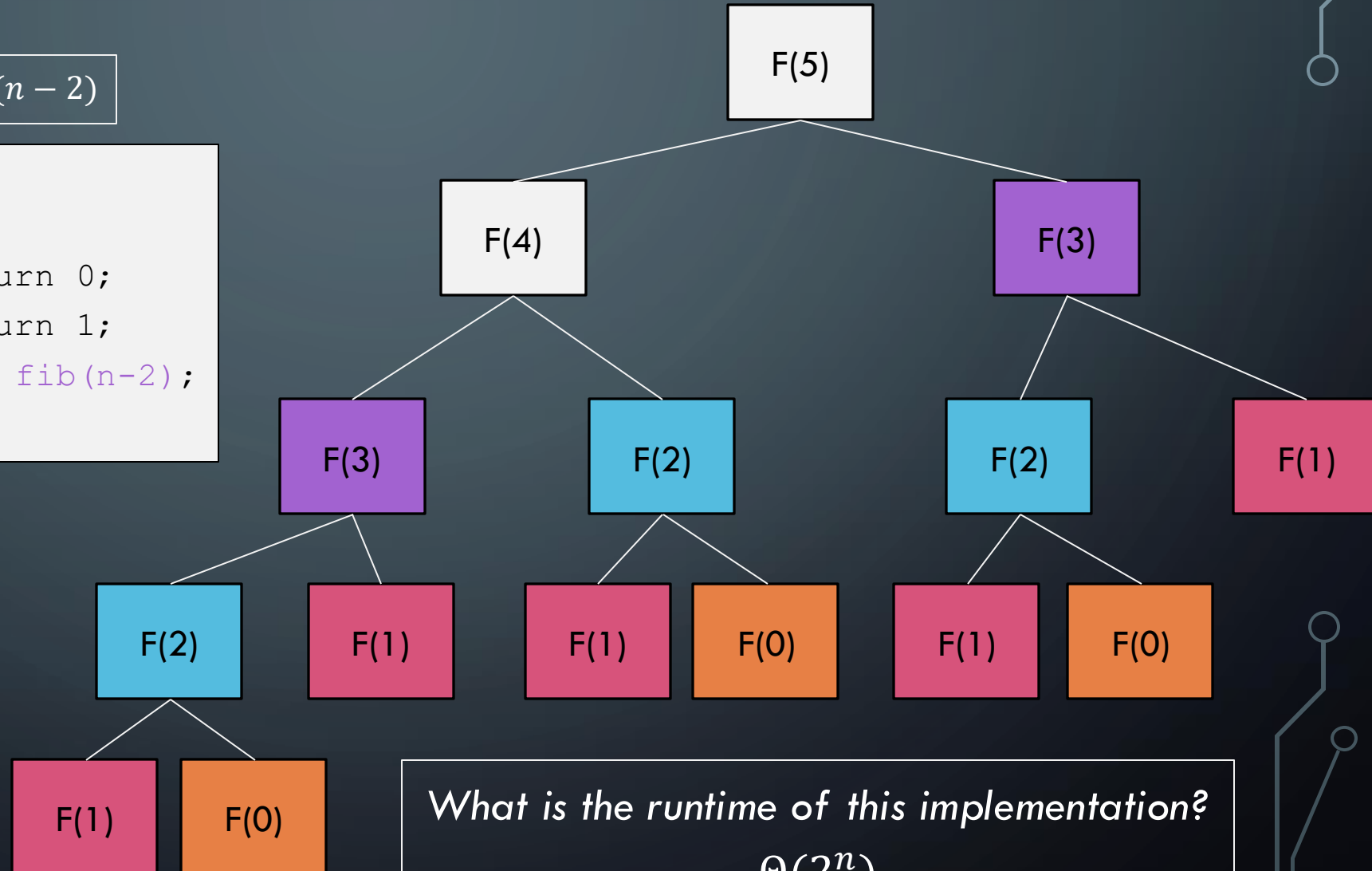
Look at how many recursive calls are repeated!

```
graph TD; F5[F(5)] --> F4[F(4)]; F5 --> F3_1[F(3)]; F4 --> F3_2[F(3)]; F4 --> F2_1[F(2)]; F3_1 --> F2_2[F(2)]; F3_1 --> F1_1[F(1)]; F3_2 --> F2_3[F(2)]; F3_2 --> F1_2[F(1)]; F2_1 --> F1_3[F(1)]; F2_1 --> F0_1[F(0)]; F2_2 --> F1_4[F(1)]; F2_2 --> F0_2[F(0)]; F2_3 --> F1_5[F(1)]; F2_3 --> F0_3[F(0)]; F1_1 --> F0_4[F(0)]; F1_1 --> F0_5[F(0)]; F1_2 --> F0_6[F(0)]; F1_2 --> F0_7[F(0)]; F1_3 --> F0_8[F(0)]; F1_3 --> F0_9[F(0)]; F1_4 --> F0_10[F(0)]; F1_4 --> F0_11[F(0)]; F1_5 --> F0_12[F(0)]; F1_5 --> F0_13[F(0)]; F1_6[F(1)] --> F0_14[F(0)]; F1_6 --> F0_15[F(0)]; F1_7[F(1)] --> F0_16[F(0)]; F1_7 --> F0_17[F(0)]; F1_8[F(1)] --> F0_18[F(0)]; F1_8 --> F0_19[F(0)]; F1_9[F(1)] --> F0_20[F(0)]; F1_9 --> F0_21[F(0)]; F1_10[F(1)] --> F0_22[F(0)]; F1_10 --> F0_23[F(0)]; F1_11[F(1)] --> F0_24[F(0)]; F1_11 --> F0_25[F(0)]; F1_12[F(1)] --> F0_26[F(0)]; F1_12 --> F0_27[F(0)]; F1_13[F(1)] --> F0_28[F(0)]; F1_13 --> F0_29[F(0)]; F1_14[F(1)] --> F0_30[F(0)]; F1_14 --> F0_31[F(0)]; F1_15[F(1)] --> F0_32[F(0)]; F1_15 --> F0_33[F(0)]; F1_16[F(1)] --> F0_34[F(0)]; F1_16 --> F0_35[F(0)]; F1_17[F(1)] --> F0_36[F(0)]; F1_17 --> F0_37[F(0)]; F1_18[F(1)] --> F0_38[F(0)]; F1_18 --> F0_39[F(0)]; F1_19[F(1)] --> F0_40[F(0)]; F1_19 --> F0_41[F(0)]; F1_20[F(1)] --> F0_42[F(0)]; F1_20 --> F0_43[F(0)]; F1_21[F(1)] --> F0_44[F(0)]; F1_21 --> F0_45[F(0)]; F1_22[F(1)] --> F0_46[F(0)]; F1_22 --> F0_47[F(0)]; F1_23[F(1)] --> F0_48[F(0)]; F1_23 --> F0_49[F(0)]; F1_24[F(1)] --> F0_50[F(0)]; F1_24 --> F0_51[F(0)]; F1_25[F(1)] --> F0_52[F(0)]; F1_25 --> F0_53[F(0)]; F1_26[F(1)] --> F0_54[F(0)]; F1_26 --> F0_55[F(0)]; F1_27[F(1)] --> F0_56[F(0)]; F1_27 --> F0_57[F(0)]; F1_28[F(1)] --> F0_58[F(0)]; F1_28 --> F0_59[F(0)]; F1_29[F(1)] --> F0_60[F(0)]; F1_29 --> F0_61[F(0)]; F1_30[F(1)] --> F0_62[F(0)]; F1_30 --> F0_63[F(0)]; F1_31[F(1)] --> F0_64[F(0)]; F1_31 --> F0_65[F(0)]; F1_32[F(1)] --> F0_66[F(0)]; F1_32 --> F0_67[F(0)]; F1_33[F(1)] --> F0_68[F(0)]; F1_33 --> F0_69[F(0)]; F1_34[F(1)] --> F0_70[F(0)]; F1_34 --> F0_71[F(0)]; F1_35[F(1)] --> F0_72[F(0)]; F1_35 --> F0_73[F(0)]; F1_36[F(1)] --> F0_74[F(0)]; F1_36 --> F0_75[F(0)]; F1_37[F(1)] --> F0_76[F(0)]; F1_37 --> F0_77[F(0)]; F1_38[F(1)] --> F0_78[F(0)]; F1_38 --> F0_79[F(0)]; F1_39[F(1)] --> F0_80[F(0)]; F1_39 --> F0_81[F(0)]; F1_40[F(1)] --> F0_82[F(0)]; F1_40 --> F0_83[F(0)]; F1_41[F(1)] --> F0_84[F(0)]; F1_41 --> F0_85[F(0)]; F1_42[F(1)] --> F0_86[F(0)]; F1_42 --> F0_87[F(0)]; F1_43[F(1)] --> F0_88[F(0)]; F1_43 --> F0_89[F(0)]; F1_44[F(1)] --> F0_90[F(0)]; F1_44 --> F0_91[F(0)]; F1_45[F(1)] --> F0_92[F(0)]; F1_45 --> F0_93[F(0)]; F1_46[F(1)] --> F0_94[F(0)]; F1_46 --> F0_95[F(0)]; F1_47[F(1)] --> F0_96[F(0)]; F1_47 --> F0_97[F(0)]; F1_48[F(1)] --> F0_98[F(0)]; F1_48 --> F0_99[F(0)]; F1_49[F(1)] --> F0_100[F(0)]; F1_49 --> F0_101[F(0)]; F1_50[F(1)] --> F0_102[F(0)]; F1_50 --> F0_103[F(0)]; F1_51[F(1)] --> F0_104[F(0)]; F1_51 --> F0_105[F(0)]; F1_52[F(1)] --> F0_106[F(0)]; F1_52 --> F0_107[F(0)]; F1_53[F(1)] --> F0_108[F(0)]; F1_53 --> F0_109[F(0)]; F1_54[F(1)] --> F0_110[F(0)]; F1_54 --> F0_111[F(0)]; F1_55[F(1)] --> F0_112[F(0)]; F1_55 --> F0_113[F(0)]; F1_56[F(1)] --> F0_114[F(0)]; F1_56 --> F0_115[F(0)]; F1_57[F(1)] --> F0_116[F(0)]; F1_57 --> F0_117[F(0)]; F1_58[F(1)] --> F0_118[F(0)]; F1_58 --> F0_119[F(0)]; F1_59[F(1)] --> F0_120[F(0)]; F1_59 --> F0_121[F(0)]; F1_60[F(1)] --> F0_122[F(0)]; F1_60 --> F0_123[F(0)]; F1_61[F(1)] --> F0_124[F(0)]; F1_61 --> F0_125[F(0)]; F1_62[F(1)] --> F0_126[F(0)]; F1_62 --> F0_127[F(0)]; F1_63[F(1)] --> F0_128[F(0)]; F1_63 --> F0_129[F(0)]; F1_64[F(1)] --> F0_130[F(0)]; F1_64 --> F0_131[F(0)]; F1_65[F(1)] --> F0_132[F(0)]; F1_65 --> F0_133[F(0)]; F1_66[F(1)] --> F0_134[F(0)]; F1_66 --> F0_135[F(0)]; F1_67[F(1)] --> F0_136[F(0)]; F1_67 --> F0_137[F(0)]; F1_68[F(1)] --> F0_138[F(0)]; F1_68 --> F0_139[F(0)]; F1_69[F(1)] --> F0_140[F(0)]; F1_69 --> F0_141[F(0)]; F1_70[F(1)] --> F0_142[F(0)]; F1_70 --> F0_143[F(0)]; F1_71[F(1)] --> F0_144[F(0)]; F1_71 --> F0_145[F(0)]; F1_72[F(1)] --> F0_146[F(0)]; F1_72 --> F0_147[F(0)]; F1_73[F(1)] --> F0_148[F(0)]; F1_73 --> F0_149[F(0)]; F1_74[F(1)] --> F0_150[F(0)]; F1_74 --> F0_151[F(0)]; F1_75[F(1)] --> F0_152[F(0)]; F1_75 --> F0_153[F(0)]; F1_76[F(1)] --> F0_154[F(0)]; F1_76 --> F0_155[F(0)]; F1_77[F(1)] --> F0_156[F(0)]; F1_77 --> F0_157[F(0)]; F1_78[F(1)] --> F0_158[F(0)]; F1_78 --> F0_159[F(0)]; F1_79[F(1)] --> F0_160[F(0)]; F1_79 --> F0_161[F(0)]; F1_80[F(1)] --> F0_162[F(0)]; F1_80 --> F0_163[F(0)]; F1_81[F(1)] --> F0_164[F(0)]; F1_81 --> F0_165[F(0)]; F1_82[F(1)] --> F0_166[F(0)]; F1_82 --> F0_167[F(0)]; F1_83[F(1)] --> F0_168[F(0)]; F1_83 --> F0_169[F(0)]; F1_84[F(1)] --> F0_170[F(0)]; F1_84 --> F0_171[F(0)]; F1_85[F(1)] --> F0_172[F(0)]; F1_85 --> F0_173[F(0)]; F1_86[F(1)] --> F0_174[F(0)]; F1_86 --> F0_175[F(0)]; F1_87[F(1)] --> F0_176[F(0)]; F1_87 --> F0_177[F(0)]; F1_88[F(1)] --> F0_178[F(0)]; F1_88 --> F0_179[F(0)]; F1_89[F(1)] --> F0_180[F(0)]; F1_89 --> F0_181[F(0)]; F1_90[F(1)] --> F0_182[F(0)]; F1_90 --> F0_183[F(0)]; F1_91[F(1)] --> F0_184[F(0)]; F1_91 --> F0_185[F(0)]; F1_92[F(1)] --> F0
```

Formula: $f(n) = f(n - 1) + f(n - 2)$

```
long fib(int n) {
    assert(n >= 0);
    if ( n == 0 ) return 0;
    if ( n == 1 ) return 1;
    return fib(n-1) + fib(n-2);
}
```

Look at how many recursive calls are repeated!



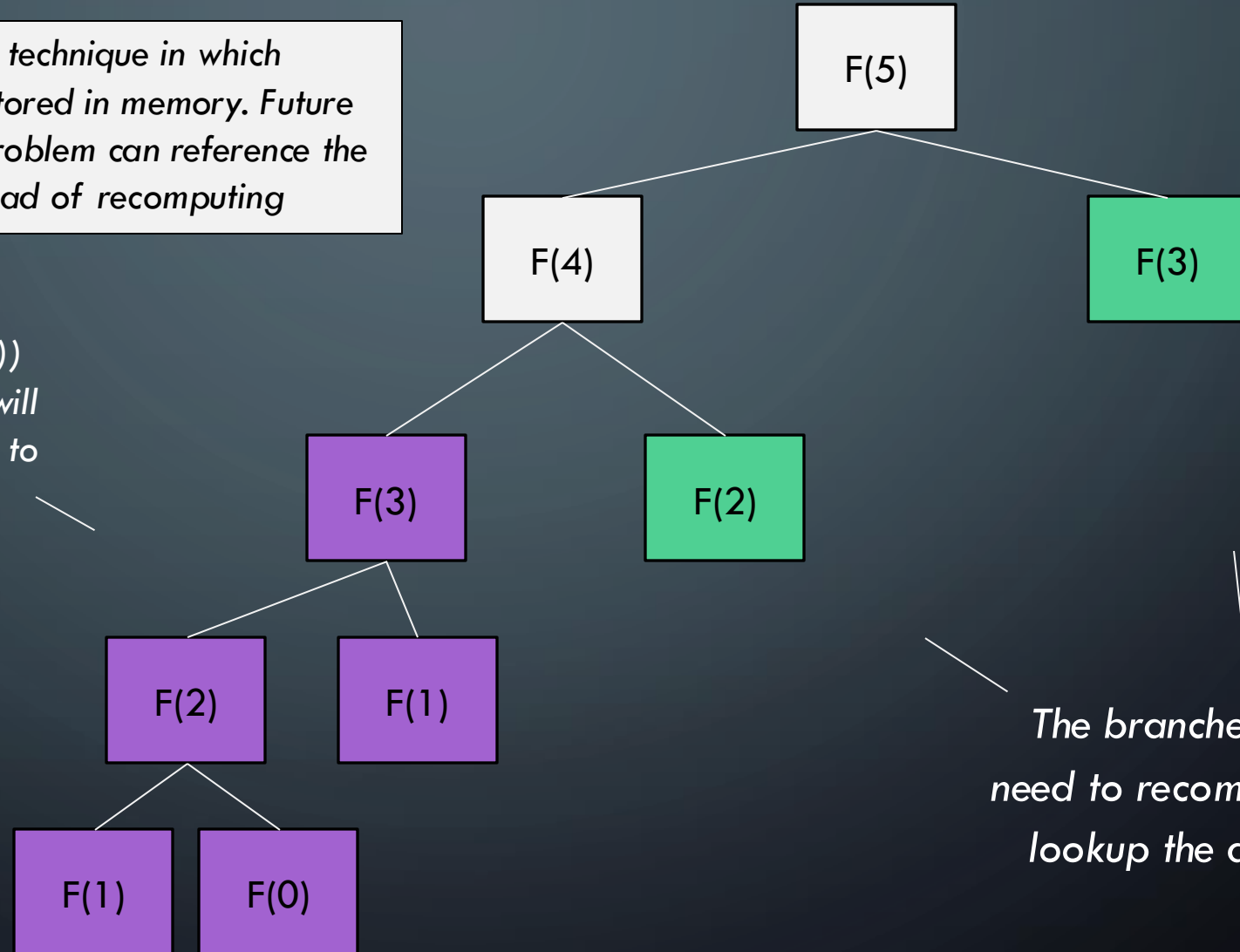
What is the runtime of this implementation?

$\Theta(2^n)$

MEMOIZATION

Memoization: A programming technique in which solutions to subproblems are stored in memory. Future invocations of that same subproblem can reference the previously stored solution instead of recomputing

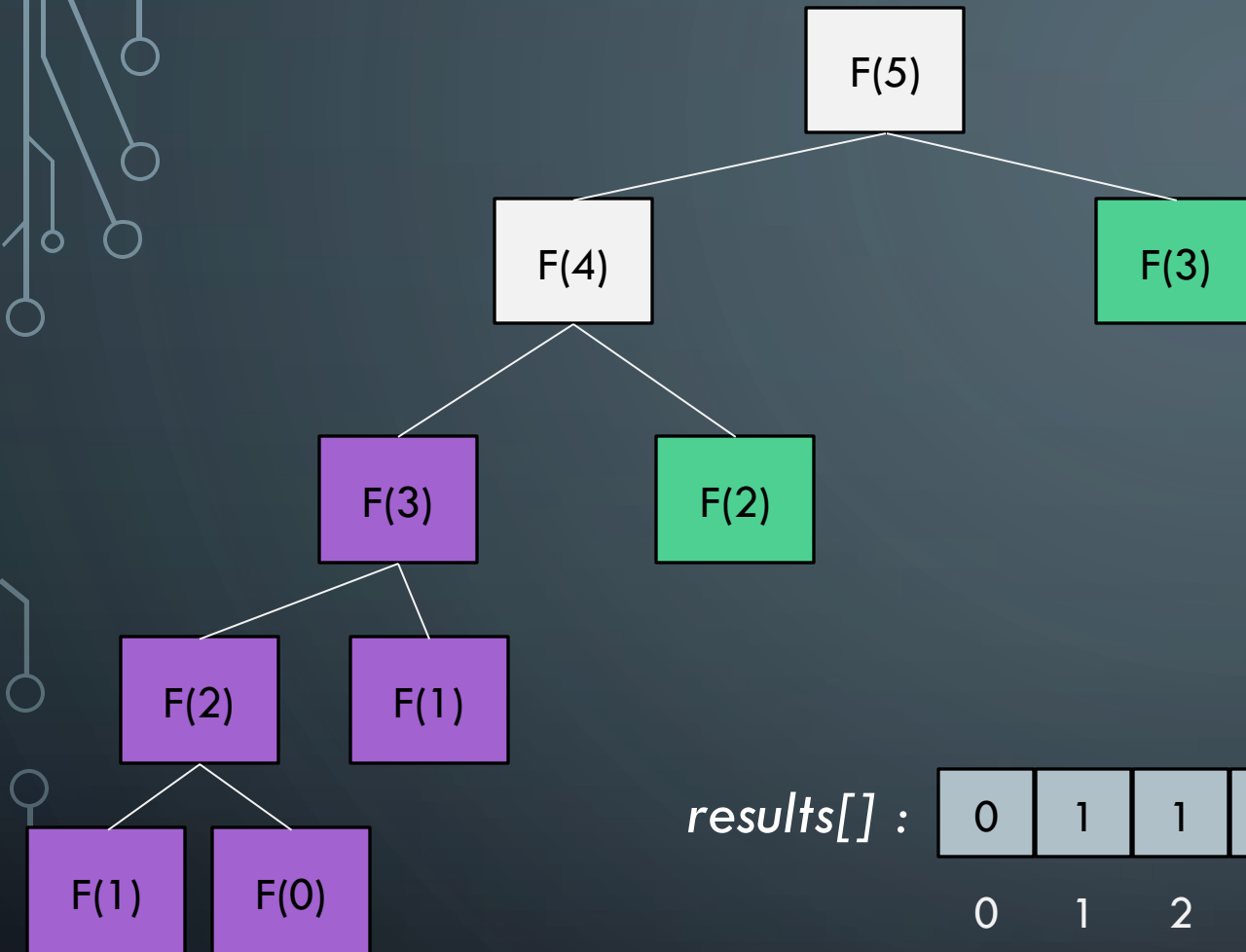
When you call a subproblem (e.g., $f(3)$) the first time, compute the answer! This will compute the answer and store solutions to $f(3)$, $f(2)$, $f(1)$, and $f(0)$ in an array!



The branches don't need to recompute, just lookup the answer!

MEMOIZATION

Key Idea: You need to be able to lookup a solution to a subproblem in constant time!



Value is -1 if that subproblem not solved yet, otherwise table contains the solution!

Last cell is the fib number we want to calculate! Here it is 11

results[] :

0	1	1	2	-1	-1	-1	-1	-1	-1	-1	-1
0	1	2	3	4	5	6	7	8	9	10	11

Memoization table. Stores solutions to subproblems!

MEMOIZATION AND FIBONACCI

```
long results[]

long fib(int n):
    results = {-1}*n
    return fibRec(n)

long fibRec(int n):

    if ( results[n] != -1 ):
        return results[n]

    long val
    if ( n == 0 || n == 1 ):
        val = n
    else
        val = fibRec(n-1) + fibRec(n-2)

    results[n] = val
    return val
```

MEMOIZATION AND FIBONACCI

```
long results[]

long fib(int n):
    results = {-1}*n
    return fibRec(n)

long fibRec(int n):

    if ( results[n] != -1 ):
        return results[n]

    long val
    if ( n == 0 || n ==1 ):
        val = n
    else
        val = fibRec(n-1)+ fibRec(n-2)

    results[n] = val
    return val
```

*Make array filled with -1 everywhere and
kick off the recursion!*

*If subproblem already solved, just return the
answer and don't do any recursion!*

Normal recursive implementation

*Before returning, store the result in results so
other subproblem calls can use it!*

MEMOIZATION AND FIBONACCI

```
long results[]

long fib(int n):
    results = {-1}*n
    return fibRec(n)

long fibRec(int n):

    if ( results[n] != -1 ):
        return results[n]

    long val
    if ( n == 0 || n ==1 ):
        val = n
    else
        val = fibRec(n-1)+ fibRec(n-2)

    results[n] = val
    return val
```

Runtime: $\Theta(n)$

Why?: Because each cell in results is computed exactly once. Future calls return in constant time.

This is called **Top-Down Dynamic Programming**

MEMOIZATION AND FIBONACCI

```
long results[]

long fib(int n):
    results = {-1}*n
    return fibRec(n)

long fibRec(int n):

    if ( results[n] != -1 ):
        return results[n]

    long val
    if ( n == 0 || n == 1 ):
        val = n
    else
        val = fibRec(n-1) + fibRec(n-2)

    results[n] = val
    return val
```

Why is this called **Top-Down Dynamic Programming**? Let's step through it!

results[] :

0	1	2	3	4	5	6	7	8	9	10	11

```
long results[]
```

```
long fib(int n):
    results = {-1}*n
    return fibRec(n)
```

```
long fibRec(int n):
```

```
if ( results[n] != -1 ):
    return results[n]
```

```
long val
if ( n == 0 || n ==1 ):
    val = n
else
    val = fibRec(n-1)+ fibRec(n-2)
```

```
results[n] = val
return val
```

Why is this called Top-Down Dynamic Programming? Let's step through it!

```
results[] :
```



BOTTOM-UP DYNAMIC PROGRAMMING

Bottom-Up Dynamic Programming: Involves solving the base cases first and then filling in the solutions in (usually) increasing order. This is typically NOT recursive

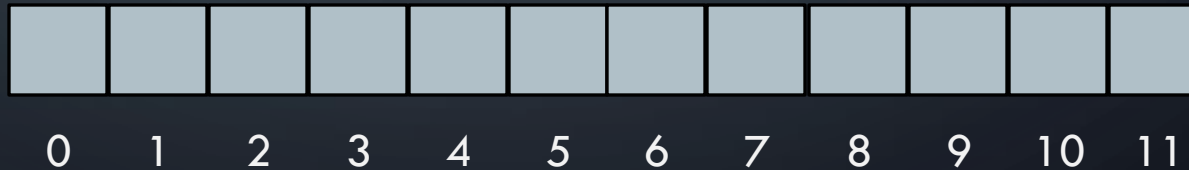
BOTTOM-UP FIBONACCI

```
fib(int n):  
    long result[n+1]  
    result[0] = 0; result[1] = 1  
  
    for i = 2 to n:  
        result[i] = result[i-1] + result[i-2]  
  
    return result[n]
```

BOTTOM-UP FIBONACCI

```
fib(int n):  
    long result[n+1]  
    result[0] = 0  
    result[1] = 1  
  
    for i = 2 to n:  
        result[i] = result[i-1] + result[i-2]  
  
    return result[n]
```

results[] :





DYNAMIC PROGRAMMING AND LOG CUTTING

PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the *sub-problems* are

Step 2:

Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the “last thing” done?)

Step 3:

Formulate a **data structure (array, table)** that can look-up solution to any sub-problem in **constant time**

Step 4:

Develop an algorithm that **loops through data structure** solving each sub-problem one at a time (either bottom-up or top-down)

LOG CUTTING

Problem Statement: Given a log of length n , and a list (of length n) of prices P (where $P[i]$ is the price of a cut of size i). Find the best way to cut the log to maximize our profit.

Price:	0	1	5	8	9	10	17	17	20	24	30
Length:	0	1	2	3	4	5	6	7	8	9	10



Select a list of lengths $l_1, \dots, l_k$ such that:
 $\sum l_i = n$ and Maximize $\sum P[l_i]$

Brute Force: $O(2^n)$

PROCESS FOR DYNAMIC PROGRAMMING

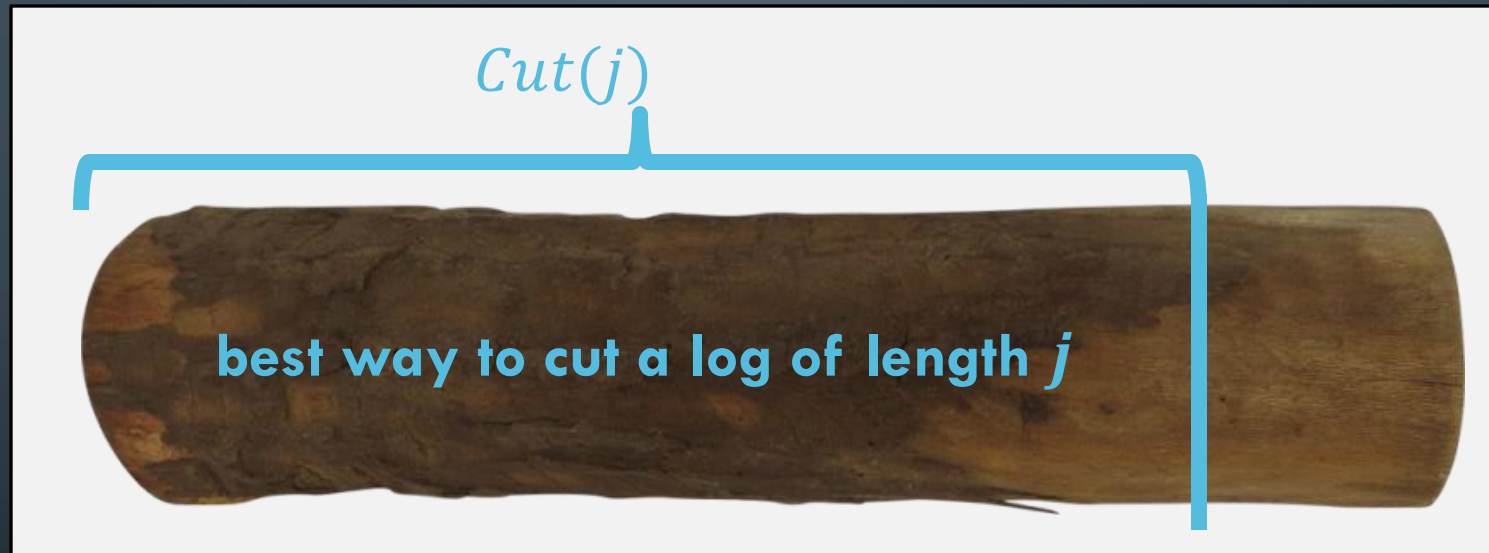
Step 1:

Recognize what the ***sub-problems*** are

1. IDENTIFY SUB-PROBLEMS

$P[i]$ = value of a cut of length i

$Cut(j)$ = value of best way to cut a log of length j



Notice that $cut(n)$ ranges from $cut(0)$ all the way to $cut(n)$

PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the *sub-problems* are

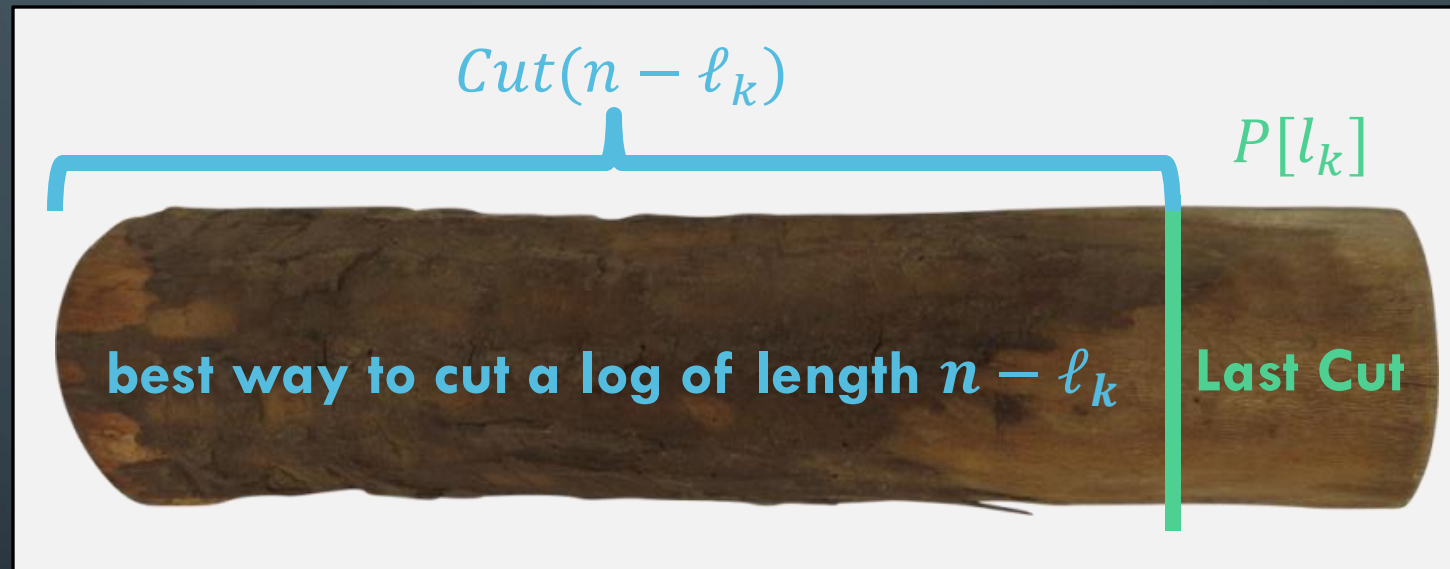
Step 2:

Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the “last thing” done?)

2. IDENTIFY RECURSIVE STRUCTURE

$P[i]$ = value of a cut of length i

$Cut(n)$ = value of best way to cut a log of length n



Key Idea: Somewhere the **very last cut was made**. The solution is the **value you get for the last cut** plus the **optimal way to cut up the rest of the log**

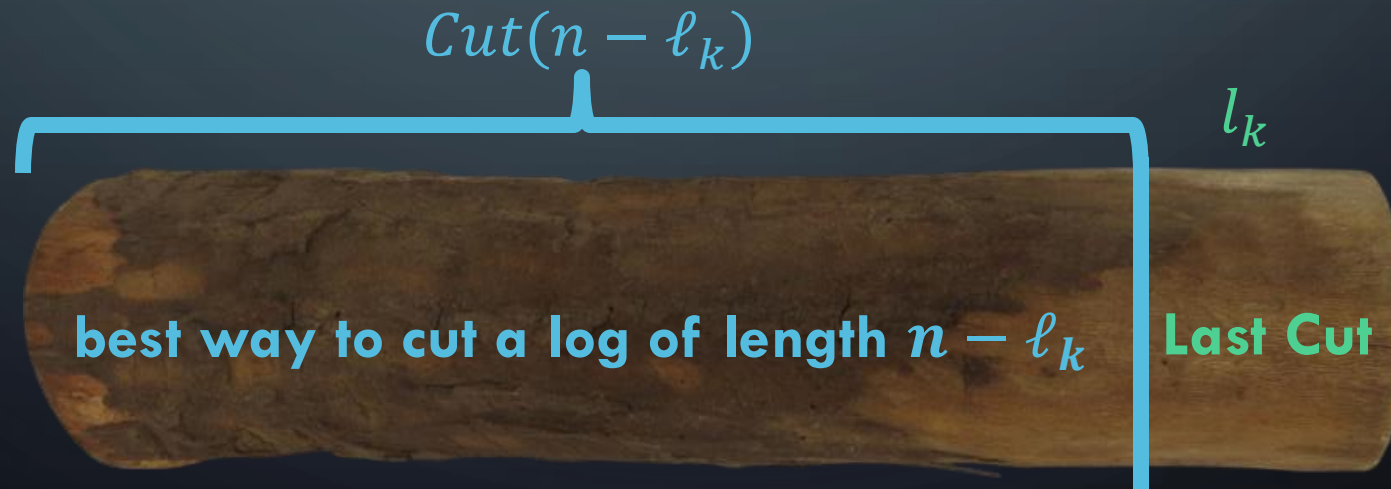
2. IDENTIFY RECURSIVE STRUCTURE

$P[i]$ = value of a cut of length i

$Cut(n)$ = value of best way to cut a log of length n

$$Cut(n) = \max \left\{ \begin{array}{l} Cut(n-1) + P[1] \\ Cut(n-2) + P[2] \\ \dots \\ Cut(0) + P[n] \end{array} \right.$$

Try every possible place
to put the last cut!



PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the *sub-problems* are

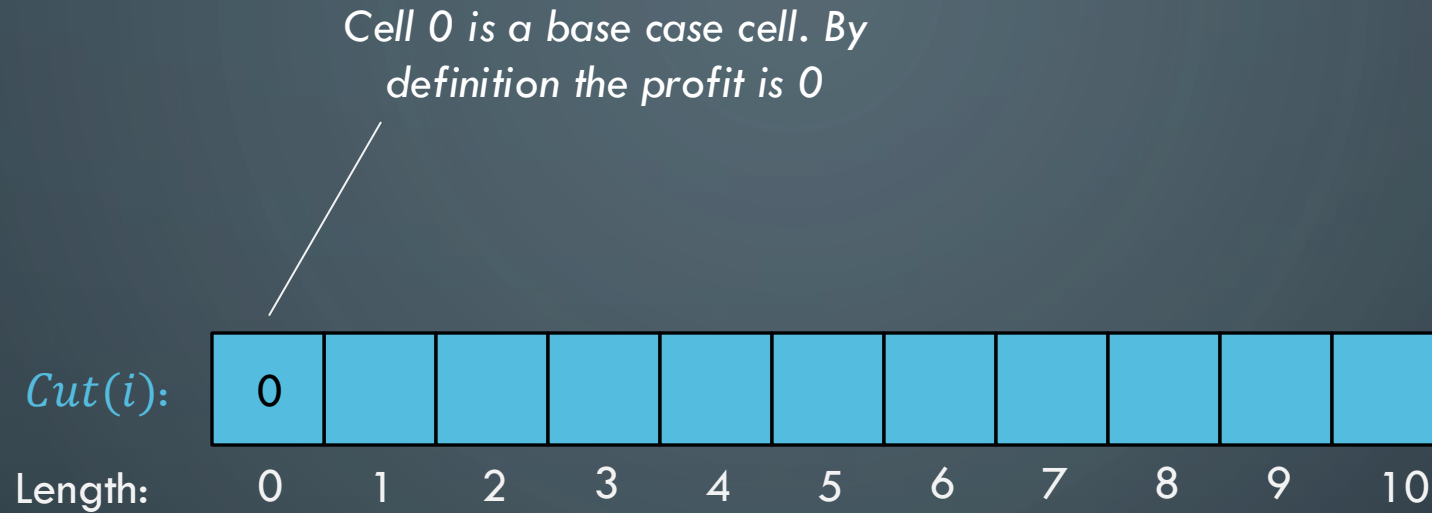
Step 2:

Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the “last thing” done?)

Step 3:

Formulate a ***data structure (array, table)*** that can look-up solution to any sub-problem in ***constant time***

3. FORMULATE A LOOK-UP DATA STRUCTURE



Each cell $Cut[i]$ stores the optimal profit you can get by cutting up a log of size i

For example, $Cut[5]$ would store the optimal profit from cutting up a log of size 5

The solution we are looking for (n) will be in this last cell!

PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the *sub-problems* are

Step 2:

Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the “last thing” done?)

Step 3:

Formulate a **data structure (array, table)** that can look-up solution to any sub-problem in **constant time**

Step 4:

Develop an algorithm that **loops through data structure** solving each sub-problem one at a time (either bottom-up or top-down)

3. ALGORITHM TO FILL IN SOLUTIONS

Solve smallest sub-problem first

Price:

1	5	8	9	10	17	17	20	24	30
---	---	---	---	----	----	----	----	----	----

Length:

1 2 3 4 5 6 7 8 9 10

Cut:

0										
---	--	--	--	--	--	--	--	--	--	--

Length:

0 1 2 3 4 5 6 7 8 9 10

```
CutProfit(n) :  
    Cut[n] = [...]  
    Cut[0] = 0  
  
    ...
```

0

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Price:

1	5	8	9	10	17	17	20	24	30
---	---	---	---	----	----	----	----	----	----

Length:

1 2 3 4 5 6 7 8 9 10

Cut:

0	1									
---	---	--	--	--	--	--	--	--	--	--

Length:

0 1 2 3 4 5 6 7 8 9 10

```
CutProfit(n) :  
    Cut[n] = [...]  
    Cut[0] = 0
```

```
for i = 1 to n:  
    ?????
```



$$Cut[1] = Cut[0] + P[1] = 1$$

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Price:

1	5	8	9	10	17	17	20	24	30
---	---	---	---	----	----	----	----	----	----

Length:

1 2 3 4 5 6 7 8 9 10

Cut:

0	1	5								
---	---	---	--	--	--	--	--	--	--	--

Length:

0 1 2 3 4 5 6 7 8 9 10

```
CutProfit(n) :
```

```
    Cut[n] = [...]
```

```
    Cut[0] = 0
```

```
    for i = 1 to n:
```

```
        ?????
```



$$Cut[2] = \max \begin{cases} Cut[1] + P[1] = 2 \\ Cut[0] + P[2] = 5 \end{cases}$$

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Price:

1	5	8	9	10	17	17	20	24	30
---	---	---	---	----	----	----	----	----	----

Length:

1 2 3 4 5 6 7 8 9 10

Cut:

0	1	5	8							
---	---	---	---	--	--	--	--	--	--	--

Length:

0 1 2 3 4 5 6 7 8 9 10

```
CutProfit(n):
```

```
  Cut[n] = [...]
```

```
  Cut[0] = 0
```

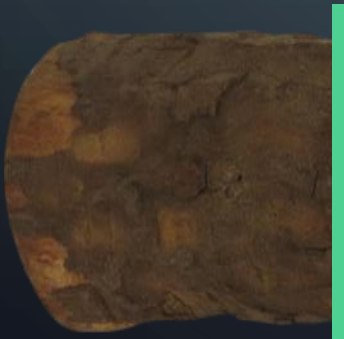
```
  for i = 1 to n:
```

```
    best = 0
```

```
    for j = 1 to i:
```

```
      best = max(Cut[i-j] + P[j])
```

```
    Cut[i] = best
```



$$Cut[3] = \max \begin{cases} Cut[2] + P[1] = 6 \\ Cut[1] + P[2] = 6 \\ Cut[0] + P[3] = 8 \end{cases}$$

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Price:

1	5	8	9	10	17	17	20	24	30
---	---	---	---	----	----	----	----	----	----

Length:

1 2 3 4 5 6 7 8 9 10

Cut:

0	1	5	8	10						
---	---	---	---	----	--	--	--	--	--	--

Length:

0 1 2 3 4 5 6 7 8 9 10

```
CutProfit(n):
```

```
  Cut[n] = [...]
```

```
  Cut[0] = 0
```

```
  for i = 1 to n:
```

```
    best = 0
```

```
    for j = 1 to i:
```

```
      best = max(Cut[i-j] + P[j])
```

```
    Cut[i] = best
```



$Cut[3] = \max$

$$Cut[3] + P[1] = 9$$

$$Cut[2] + P[2] = 10$$

$$Cut[1] + P[3] = 9$$

$$Cut[0] + P[4] = 9$$

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

Price:

1	5	8	9	10	17	17	20	24	30
---	---	---	---	----	----	----	----	----	----

Length:

1 2 3 4 5 6 7 8 9 10

Cut:

0	1	5	8	10	13	17	18	22	25	30
---	---	---	---	----	----	----	----	----	----	----

Length:

0 1 2 3 4 5 6 7 8 9 10

Solution is here in the last cell, so just return it!

```
CutProfit(n):  
    Cut[] = new [n+1]  
    Cut[0] = 0  
  
    for i = 1 to n:  
        best = 0  
        for j = 1 to i:  
            best = max(Cut[i-j] + P[j])  
        Cut[i] = best  
  
    return Cut[n]
```

3. ALGORITHM TO FILL IN SOLUTIONS

Then the next largest subproblem

<i>Cut:</i>	0	1	5	8	10	13	17	18	22	25	30
Length:	0	1	2	3	4	5	6	7	8	9	10

```
CutProfit(n):  
    Cut[] = new [n+1]  
    Cut[0] = 0  
  
    for i = 1 to n:  
        best = 0  
        for j = 1 to i:  
            best = max(Cut[i-j] + P[j])  
        Cut[i] = best  
  
    return Cut[n]
```

Runtime is $\Theta(n^2)$

PROCESS FOR DYNAMIC PROGRAMMING

Step 1:

Recognize what the *sub-problems* are

Step 2:

Identify the recursive structure of the problem in terms of its sub-problems (At the top level, what is the “last thing” done?)

Step 3:

Formulate a ***data structure (array, table)*** that can look-up solution to any sub-problem in ***constant time***

Step 4:

Develop an algorithm that ***loops through data structure*** solving each sub-problem one at a time (either bottom-up or top-down)



HOW TO FIND THE ACTUAL CUTS

EXTRA: FINDING THE ACTUAL SOLUTION

This gives the value of the solution, what about the actual list of where to cut the log?

<i>Cut:</i>	0	1	5	8	10	13	17	18	22	25	30
Length:	0	1	2	3	4	5	6	7	8	9	10

Idea: Keep track of where the last cut was for each subproblem!

EXTRA: FINDING THE ACTUAL SOLUTION

This gives the value of the solution, what about the actual list of where to cut the log?

Cut:

0	1	5	8	10	13	17	18	22	25	30
---	---	---	---	----	----	----	----	----	----	----

Length:

0 1 2 3 4 5 6 7 8 9 10

Choices:

0	1	2								
---	---	---	--	--	--	--	--	--	--	--

Length:

0 1 2 3 4 5 6 7 8 9 10

$$Cut[2] = \max \begin{cases} Cut[1] + P[1] = 2 \\ Cut[0] + P[2] = 5 \end{cases}$$

Idea: Keep track of where the last cut was for each subproblem!

e.g., Choices[2]=2 because last cut was a piece of size 2

EXTRA: FINDING THE ACTUAL SOLUTION

Cut:

0	1	5	8	10	13	17	18	22	25	30
---	---	---	---	----	----	----	----	----	----	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

Choices:

0	1	2	3	2	2	6	1	2	3	10
---	---	---	---	---	---	---	---	---	---	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

So in this case, the optimal solution is to just sell the entire log of size 10

EXTRA: FINDING THE ACTUAL SOLUTION

Cut:

0	1	5	8	10	13	17	18	22	25	30
---	---	---	---	----	----	----	----	----	----	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

Choices:

0	1	2	3	2	2	6	1	2	3	10
---	---	---	---	---	---	---	---	---	---	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

```
CutProfit(n):
```

```
    Cut[] = new [n+1]
```

```
    Choices[] = new [n+1]
```

```
    Cut[0] = 0
```

```
    Choices[0] = 0
```

```
    for i = 1 to n:
```

```
        best = 0
```

```
        for j = 1 to i:
```

```
            if best < Cut[i-j]+P[j]:
```

```
                best = Cut[i-j]+P[j]
```

```
                Choices[i] = j
```

```
        Cut[i] = best
```

```
    return Cut[n]
```

EXTRA: FINDING THE ACTUAL SOLUTION

Cut:

0	1	5	8	10	13	17	18	22	25	30
---	---	---	---	----	----	----	----	----	----	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

Choices:

0	1	2	3	2	2	6	1	2	3	10
---	---	---	---	---	---	---	---	---	---	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

Let's use this choices array to figure out how to achieve a profit of 25 with a log of size 9?

EXTRA: FINDING THE ACTUAL SOLUTION

Cut:

0	1	5	8	10	13	17	18	22	25	30
---	---	---	---	----	----	----	----	----	----	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

Choices:

0	1	2	3	2	2	6	1	2	3	10
---	---	---	---	---	---	---	---	---	---	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

Solution: 3

Left: 6

This 3 means that we cut off and sell a log piece of size 3

Log of size 6 left so backtrack to index 6

EXTRA: FINDING THE ACTUAL SOLUTION

Cut:

0	1	5	8	10	13	17	18	22	25	30
---	---	---	---	----	----	----	----	----	----	----

Length:

0 1 2 3 4 5 6 7 8 9 10

Choices:

0	1	2	3	2	2	6	1	2	3	10
---	---	---	---	---	---	---	---	---	---	----

Length:

0 1 2 3 4 5 6 7 8 9 10

Solution: 3 6

Left: 0

This 6 means that we cut off and sell a log piece of size 6

Log of size 0 left so backtrack to index 0

EXTRA: FINDING THE ACTUAL SOLUTION

Cut:

0	1	5	8	10	13	17	18	22	25	30
---	---	---	---	----	----	----	----	----	----	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

Choices:

0	1	2	3	2	2	6	1	2	3	10
---	---	---	---	---	---	---	---	---	---	----

Length:

0	1	2	3	4	5	6	7	8	9	10
---	---	---	---	---	---	---	---	---	---	----

Solution: 3 6

Once we hit index 0 we are done.
Solution is to split log into sizes 3 and 6 and sell those two pieces!

EXTRA: FINDING THE ACTUAL SOLUTION

<i>Cut:</i>	0	1	5	8	10	13	17	18	22	25	30
Length:	0	1	2	3	4	5	6	7	8	9	10

<i>Choices:</i>	0	1	2	3	2	2	6	1	2	3	10
Length:	0	1	2	3	4	5	6	7	8	9	10

Solution: 3 6

```
BackTrack(Choices) :  
    i = n  
    while i > 0:  
        print Choices[i]  
        i = i - Choices[i]
```

With dynamic programming, it is VERY common to have the array store the value of solutions only and then use backtracking to compute the solution itself!

WHAT DID WE LEARN?

- What is dynamic programming?
General Approach
- First Example: Log cutting